



Custom Time Series Forecasting in R: Predictors, Filters, Augmentation, Normalizers

Custom Examples in tspredit

This section shows how to extend `tspredit` without abandoning the package's standard workflow. The forecasting pipeline remains stable while the user connects new components through a small, well-defined integration contract.

01

Custom Predictor

Create a predictor based on `RSNNS::mlp` and tune it via `ts_regsw`.

03

Custom Augmentation

Create a magnitude warping operator for sliding windows.

02

Custom Filter

Implement a median filter for spike-type noise.

04

Custom Normalization

Build an adaptive divisive normalization with moving statistics.

Defining the Class and S3 Methods

Objective

Create a custom predictor based on `RSNNS::mlp`, integrating it into the `tspredict` pipeline through the S3 contract — implementing the `do_fit` and `do_predict` methods — without altering the package's standard flow.

Step by step

- The constructor calls `ts_regsw` and stores the hyperparameters in the object.
- `do_fit()` trains the MLP network via `RSNNS::mlp` with linear output (`linOut = TRUE`).
- `do_predict()` returns the predictions as a numeric vector.
- The methods are registered with `registerS3method()` for automatic integration into the pipeline.

```
ts_mlp_rsnns_custom <- function(  
  preprocess = ts_norm_gminmax(),  
  input_size = 4,  
  size = 5,  
  learn_rate = 0.1,  
  maxit = 200) {  
  obj <- ts_regsw(  
    preprocess = preprocess,  
    input_size = input_size)  
  obj$size <- size  
  obj$learn_rate <- learn_rate  
  obj$maxit <- maxit  
  class(obj) <- append(  
    "ts_mlp_rsnns_custom", class(obj))  
  obj  
}
```

```
registerS3method("do_fit", "ts_mlp_rsnns_custom"  
  do_fit.ts_mlp_rsnns_custom)  
registerS3method("do_predict", "ts_mlp_rsnns_custom",  
  do_predict.ts_mlp_rsnns_custom)
```

```
do_fit.ts_mlp_rsnns_custom <- function(obj, x, y) {  
  obj$model <- RSNNS::mlp(  
    x = as.matrix(x),  
    y = as.matrix(y),  
    size = obj$size,  
    learnFuncParams = c(obj$learn_rate),  
    maxit = obj$maxit,  
    linOut = TRUE)  
  obj  
}
```

```
do_predict.ts_mlp_rsnns_custom <- function(obj, x)  
{  
  as.numeric(predict(obj$model, as.matrix(x)))  
}
```

Fitting, Prediction and Evaluation

Objective

Demonstrate the full usage of the custom predictor `ts_mlp_rsnns_custom` in the standard `tspredict` pipeline, from data preparation to training and test metric evaluation.

Step by step

- `ts_data()` creates sliding windows; `ts_sample()` splits into train and test.
- `ts_projection()` divides each window into inputs and outputs.
- `fit()` and `predict()` are called exactly as with any native predictor in the package.
- Training metrics: **MSE 0.0001, R² 0.9998**.
Test metrics: **MSE 0.0015, R² 0.9871**.

```
set.seed(123)
data(tsd)

ts <- ts_data(tsd$y, 10)
samp <- ts_sample(ts, test_size = 5)
io_train <- ts_projection(samp$train)
io_test <- ts_projection(samp$test)

model <- ts_mlp_rsnns_custom(
  preprocess = ts_norm_gminmax(),
  input_size = 4,
  size       = 5,
  learn_rate = 0.05,
  maxit      = 300)

model <- fit(model,
  x = io_train$input,
  y = io_train$output)
```

```
# Training evaluation
adjust <- as.vector(predict(model,
  io_train$input))
train_eval <- evaluate(model,
  as.vector(io_train$output), adjust)
train_eval$metrics
## mse      smape    R2
## 0.0001039913 0.02831342 0.999791

# Test prediction (5 steps ahead)
prediction <- as.vector(predict(model,
  x = io_test$input[1:1, ], steps_ahead = 5))
test_eval <- evaluate(model,
  as.vector(io_test$output), prediction)
test_eval$metrics
## mse      smape    R2
## 0.00149765 0.1729205 0.9870647
```

Median Filter for Spike Removal

Objective

Implement a custom median filter for removing spike-type noise from time series, inheriting from `dal_transform()` and implementing only the `transform()` method.

Step by step

- The constructor ensures that `k` is always odd (required by `runmed`).
- Inherits from `dal_transform()`; only `transform()` needs to be implemented when there is no fitted state.
- Spikes are manually injected at positions 10, 20, and 30 to make the filter effect visible.
- After filtering, the cleaned series is passed directly to the standard predictive pipeline.

```
library(daltoolbox)
library(tspredit)
library(ggplot2)

# Definition of the custom component
ts_fil_median_custom <- function(k = 5) {
  if (k %% 2 == 0) k <- k + 1
  obj <- daltoolbox::dal_transform()
  obj$k <- k
  class(obj) <- append(
    "ts_fil_median_custom", class(obj))
  obj
}

transform.ts_fil_median_custom <- function(obj, data,
...) {
  result <- stats::runmed(
    as.numeric(data), k = obj$k, endrule = "keep")
  result[is.na(result)] <-
    as.numeric(data)[is.na(result)]
  result
}
```

```
# Spike injection and filter application
data(tsd)
y_noisy <- tsd$y
y_noisy[c(10, 20, 30)] <-
  y_noisy[c(10, 20, 30)] + c(1, -1.2, 1.1)

filter_custom <- ts_fil_median_custom(k = 5)
y_filtered <- transform(filter_custom, y_noisy)

plot_ts_pred(y = y_noisy, yadj = y_filtered)
```

Using the Filter in the Predictive Pipeline

Objective

Integrate the series filtered by the median filter into the standard predictive pipeline of `tspredict`, demonstrating that the filter acts as an *upstream* preprocessing step without changing the forecasting interface.

Step by step

- The filtered series replaces the original series as input to `ts_data()`.
- The rest of the pipeline (`ts_sample`, `ts_projection`, `fit`, `predict`, `evaluate`) remains identical.
- The filter acts as an upstream preprocessing step, without changing the predictive interface.

```
# Filtered series → windows → sampling
ts_filtered <- ts_data(y_filtered, 10)
samp      <- ts_sample(ts_filtered, test_size = 5)
io_train  <- ts_projection(samp$train)
io_test   <- ts_projection(samp$test)
```

```
# Standard MLP model on filtered data
model <- ts_mlp(
  ts_norm_gminmax(),
  input_size = 4,
  size       = 4,
  decay      = 0,
  maxit      = 1000)
```

```
model <- fit(model,
  x = io_train$input,
  y = io_train$output)

prediction <- as.vector(predict(model,
  x = io_test$input[1:1, ], steps_ahead = 5))

evaluate(model,
  as.vector(io_test$output),
  prediction)$metrics
## mse      smape   R2
## 0.07889561 1.215668 0.3185745
```

Magnitude Warping for Sliding Windows

Objective

Implement a custom **magnitude warping** operator for sliding window augmentation, multiplying each lag profile by a random spline curve to create amplitude variations without altering the temporal ordering.

Step by step

- The constructor inherits from `dal_transform()` and stores `sigma`, `knots`, and `preserve_data`.
- `warp_one()` generates a smooth curve via cubic spline and multiplies the lags of each window.
- This creates local amplitude variations while preserving temporal order.
- `preserve_data = TRUE` concatenates the original windows with the augmented ones, doubling the training set.

```
ts_aug_magwarp_custom <- function(  
  sigma = 0.2, knots = 4,  
  preserve_data = TRUE) {  
  obj <- daltoolbox::dal_transform()  
  obj$sigma <- sigma  
  obj$knots <- knots  
  obj$preserve_data <- preserve_data  
  class(obj) <- append(  
    "ts_aug_magwarp_custom", class(obj))  
  obj  
}
```

```
warp_one <- function(row) {  
  p <- length(row) - 1  
  if (p < 2) return(row)  
  anchor_x <- seq(1, p, length.out = obj$knots)  
  anchor_y <- stats::rnorm(  
    obj$knots, mean = 1, sd = obj$sigma)  
  curve <- stats::spline(  
    anchor_x, anchor_y,  
    xout = 1:p, method = "natural")$y  
  result <- row  
  result[1:p] <- row[1:p] * curve  
  result  
}
```

```
transform.ts_aug_magwarp_custom <- function(obj, data,  
  ...) {  
  augmented <- t(apply(data, 1, warp_one))  
  augmented <- adjust_ts_data(augmented)  
  attr(augmented, "idx") <- 1:nrow(data)  
  if (obj$preserve_data) {  
    idx <- c(1:nrow(data),  
      attr(augmented, "idx"))  
    augmented <- rbind(data, augmented)  
    augmented <- adjust_ts_data(augmented)  
    attr(augmented, "idx") <- idx  
  }  
  augmented  
}
```

Applying Augmentation in the Pipeline

Objective

Apply the magnitude warping operator to the training set, doubling the number of available windows (from 32 to 64), and train an MLP model on the augmented data to evaluate the impact on predictive performance.

Step by step

- Augmentation doubles the training rows: from **32 to 64 windows**.
- `adjust_ts_data()` ensures the format of the augmented windows is compatible with the pipeline.
- The test set remains untouched — only the training set is enriched.
- Result: **MSE 0.037**, **R² 0.684** with expanded training data.

```
set.seed(123)
data(tsd)

train_windows <- ts_data(tsd$y, 10)
augment_custom <- ts_aug_magwarp_custom(
  sigma = 0.15, knots = 4)
train_aug <- transform(augment_custom,
  train_windows)

data.frame(
  original_rows = nrow(train_windows),
  augmented_rows = nrow(train_aug))
## original_rows augmented_rows
## 32 64

# Pipeline with augmented data
samp <- ts_sample(train_windows, test_size =
5)
train_aug <- transform(augment_custom,
  samp$train)
train_aug <- adjust_ts_data(train_aug)
```

```
io_train <- ts_projection(train_aug)
io_test <- ts_projection(samp$test)

model <- ts_mlp(ts_norm_gminmax(),
  input_size = 4, size = 4,
  decay = 0, maxit = 1000)
model <- fit(model,
  x = io_train$input, y = io_train$output)

prediction <- as.vector(predict(model,
  x = io_test$input[1:1, ], steps_ahead = 5))
evaluate(model,
  as.vector(io_test$output), prediction)$metrics
## mse smape R2
## 0.03657729 0.8413674 0.6840801
```


Adaptive Divisive Normalization – Definition

Objective

Implement a custom adaptive divisive normalizer that divides each window by its moving average before global min-max rescaling, allowing the model to learn relative variations instead of absolute values — especially useful for strictly positive series such as GDP.

Step by step

- The constructor defines the moving average (`ma`) using the last `nw` columns of the window.
- `fit()` divides each window by the moving average and applies global min-max, storing `gmin` and `gmax`.
- `eps` prevents division by zero when the mean is close to zero.
- Useful for strictly positive series (e.g., GDP), where the model should focus on relative level, not absolute magnitude.

```
ts_norm_an_div_custom <- function(  
  outliers = daltoolbox::outliers_boxplot(),  
  nw = 0, eps = 1e-8) {  
  obj <- daltoolbox::dal_transform()  
  obj$outliers <- outliers  
  obj$nw <- nw  
  obj$eps <- eps  
  obj$an_mean <- mean  
  obj$ma <- function(obj, data, func) {  
    data <- unclass(as.matrix(data))  
    if (obj$nw != 0) {  
      cols <- ncol(data) - ((obj$nw - 1):0)  
      data <- data[, cols, drop = FALSE]  
    }  
    apply(data, 1, func, na.rm = TRUE)  
  }  
  class(obj) <- append(  
    "ts_norm_an_div_custom", class(obj))  
  obj  
}
```

```
fit.ts_norm_an_div_custom <- function(obj, data, ...) {  
  input <- data[, 1:(ncol(data) - 1)]  
  an <- obj$ma(obj, input, obj$an_mean)  
  denom <- ifelse(abs(an) < obj$eps, obj$eps, an)  
  data <- sweep(data, 1, denom, "/")  
  if (!is.null(obj$outliers)) {  
    out <- fit(obj$outliers, data)  
    data <- transform(out, data)  
  }  
  obj$gmin <- min(data)  
  obj$gmax <- max(data)  
  obj  
}
```

Transform and Inverse Transform

Goal

Implement the `transform()` and `inverse_transform()` methods of the divisive adaptive normalizer, ensuring that the denominator (moving average) is stored as an attribute to allow correct scale inversion on predictions.

Step by step

- `transform()` stores the denominator as an attribute for later use in inversion.
- `inverse_transform()` reverts the min-max and multiplies by the original denominator, recovering the real scale.

```
transform.ts_norm_an_div_custom <- function(
  obj, data, x = NULL, ...) {
  if (!is.null(x)) {
    denom <- attr(data, "an_denom")
    x <- x / denom
    x <- (x - obj$gmin) / (obj$gmax - obj$gmin)
    return(x)
  }
  an <- obj$ma(obj, data, obj$an_mean)
  denom <- ifelse(abs(an) < obj$eps, obj$eps,
an)
  data <- sweep(data, 1, denom, "/")
  data <- (data - obj$gmin) / (obj$gmax -
obj$gmin)
  attr(data, "an_mean") <- an
  attr(data, "an_denom") <- denom
  data
}
```

```
inverse_transform.ts_norm_an_div_custom
<- function(
  obj, data, x = NULL, ...) {
  denom <- attr(data, "an_denom")
  if (!is.null(x)) {
    x <- x * (obj$gmax - obj$gmin) + obj$gmin
    x <- x * denom
    return(x)
  }
  data <- data * (obj$gmax - obj$gmin) +
obj$gmin
  data <- data * denom
  attr(data, "an_denom") <- denom
  data
}
```

Using the Normalizer in the Pipeline

Goal

Demonstrate the direct integration of the divisive adaptive normalizer into `ts_mlp()` via the `preprocess` argument, applying it to GDP data and evaluating predictive performance with test metrics.

Step by step

- The normalizer is passed directly as `preprocess` to `ts_mlp()` — transparent integration.
- The fit, predict, and evaluate pipeline remains identical to any other native normalizer.
- Result on GDP: SMAPE 0.033, R^2 0.836.

```
# Using the normalizer in the pipeline with
GDP data
data(gdp)
ts <- ts_data(gdp$y, 10)
samp <- ts_sample(ts, test_size = 5)
io_train <- ts_projection(samp$train)
io_test <- ts_projection(samp$test)
```

```
model <- ts_mlp(
  preprocess = ts_norm_an_div_custom(nw =
3),
  input_size = 4, size = 4,
  decay = 0, maxit = 1000)
model <- fit(model,
  x = io_train$input, y = io_train$output)
```

```
prediction <- as.vector(predict(model,
  x = io_test$input[1:1, ], steps_ahead = 5))
evaluate(model,
  as.vector(io_test$output),
  prediction)$metrics
## mse      smape    R2
## 864812660441 0.03338561 0.8358774
```

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

